

ORDENACION, BUSQUEDA Y MEZCLA (INTERNA)

vendedor \ modelo	1	2	3	4	...	15
1	4	8	1	4		
2	12	4	25	14		
3	15	3	4	7		
10						

En un tablero de ajedrez la reina puede atacar cualquier pieza que esté en la misma fila, columna o diagonal que la reina. El problema de las n reinas es posicionar n reinas en un tablero $n \times n$, de modo que ninguna reina pueda atacar a ninguna otra. Escribir un programa que solucione este problema para un valor dado de n .

CONTENIDO

- 12.1. Introducción
- 12.2. Ordenación
- 12.3. Ordenación por burbuja
- 12.4. Ordenación por selección
- 12.5. Ordenación por inserción
- 12.6. Ordenación Shell
- 12.7. Búsqueda lineal
- 12.8. Búsqueda binaria
- 12.9. Mezcla

RESUMEN PROBLEMAS

Los computadores emplean gran parte de su tiempo en operaciones de ordenación, de búsqueda y mezcla. Los arrays (vectores y tablas) se utilizan con mucha frecuencia para almacenar datos, por ello los algoritmos para el diseño de estas operaciones son fundamentales y se denominan a las operaciones *internas*, debido a que los arrays guardan sus datos de modo temporal en memoria interna y desaparecen cuando se apaga la computadora.

Los métodos de ordenación, búsqueda y mezcla son numerosos; en este capítulo se consideran algunos de los más eficientes. Su importancia reside en el hecho de que su análisis y algoritmos servirán también, en gran medida, para realizar las mismas operaciones con registros, archivos y estructuras dinámicas de datos.

12.1. INTRODUCCION

Tres operaciones muy importantes en programación de computadores son: *ordenación*, *búsqueda* y *mezcla*; son esenciales para un gran número de programas de proceso de datos y se estima que en estas operaciones los computadores por término medio gastan gran parte de su tiempo. La búsqueda, ordenación y mezcla son también procesos que las personas se encuentran normalmente en sus vidas diarias. Considérese, por ejemplo, el proceso de encontrar una palabra en un diccionario o un nombre en una guía o listado

de teléfonos. La *búsqueda* de un elemento específico se simplifica considerablemente por el hecho de que las palabras en el diccionario y los nombres en la guía telefónica están *ordenados* o *clasificados* en orden alfabético. Asimismo, la operación de mezclar datos de dos listas o conjuntos de datos en una sola lista suele ser una operación frecuente.

En este capítulo tratamos los métodos más usuales de ordenación, búsqueda y mezcla, relativos a listas o vectores (arrays), ya que si bien estas operaciones se aplican sobre otras estructuras de datos como registros o archivos, su mayor aplicación está casi siempre asociada a los vectores, y, por otra parte, la comprensión de los algoritmos en vectores —extrapolables a otras estructuras— es más fácil.

12.2. ORDENACION

La **ordenación** o **clasificación** de datos (*sort* en inglés) es una operación consistente en disponer un conjunto —estructura— de datos en algún determinado orden con respecto a uno de los *campos* de elementos del conjunto. Por ejemplo, cada elemento del conjunto de datos de una guía telefónica tiene un campo nombre, un campo dirección y un campo número de teléfono; la guía telefónica está dispuesta en orden alfabético de nombres. Los elementos numéricos se pueden ordenar en orden creciente o decreciente de acuerdo al valor numérico del elemento. En terminología de ordenación, el elemento por el cual está ordenado un conjunto de datos (o se está buscando) se denomina *clave*.

Una colección de datos (*estructura*) puede ser almacenada en un *archivo*, un *array* (vector o *tabla*), un *array de registros*, una *lista enlazada* o un *árbol*. Cuando los datos están almacenados en un array, una lista enlazada o un árbol, se denomina *ordenación interna*. Si los datos están almacenados en un archivo, el proceso de ordenación se llama *ordenación externa*.

Una *lista* se dice que *está ordenada por la clave k* si la lista está en orden ascendente o descendente con respecto a esta clave. La lista se dice que está en *orden ascendente* si:

$$i < j \quad \text{implica que} \quad K[i] \leq K[j]$$

y se dice que está en *orden descendente* si:

$$i > j \quad \text{implica que} \quad K[i] \geq K[j]$$

para todos los elementos de la lista. Por ejemplo, para una guía telefónica, la lista está clasificada en orden ascendente por el campo clave *k*, donde *k[i]* es el nombre del abonado (apellidos, nombre).

4	5	14	21	32	45	<i>orden ascendente</i>
75	70	35	16	14	12	<i>orden descendente</i>
Zacarias	Rodriguez	Martinez	Lopez	Garcia		<i>orden descendente</i>

Los métodos (algoritmos) de ordenación son numerosos, por ello se debe prestar especial atención en su elección. ¿Cómo se sabe cuál es el mejor algoritmo? La *eficiencia* es el factor que mide la calidad y rendimiento de un algoritmo. En el caso de la operación de ordenación, dos criterios se suelen seguir a la hora de decidir qué algoritmo —de entre los que resuelven la ordenación— es el más eficiente: 1) *tiempo menor de ejecución en computadora*; 2) *menor número de instrucciones*. Sin embargo, no siempre es fácil

efectuar estas medidas: puede no disponerse de instrucciones para medida de tiempo —aunque no sea este el caso de Turbo Pascal—, y las instrucciones pueden variar, dependiendo del lenguaje y del propio estilo del programador. Por esta razón, el mejor criterio para medir la eficiencia de un algoritmo es aislar una operación específica clave en la ordenación y contar el número de veces que se realiza. Así, en el caso de los algoritmos de ordenación, se utilizará como medida de su eficiencia el número de comparaciones entre elementos efectuados. El algoritmo de ordenación A será más eficiente que el B, si requiere menor número de comparaciones. Así, en el caso de ordenar los elementos de un vector, el número de comparaciones será *función* del número de elementos (n) del vector (array). Por consiguiente, se puede expresar el número de comparaciones en términos de n (por ejemplo, $n + 4$, o bien n^2) en lugar de números enteros (por ejemplo, 325).

En todos los métodos de este capítulo, normalmente —para comodidad del lector— se utiliza el orden ascendente sobre vectores o listas (arrays unidimensionales).

Los métodos de ordenación se suelen dividir en dos grandes grupos:

- directos burbuja, selección, inserción
- indirectos (avanzados) Shell, ordenación rápida, ordenación por mezcla

En el caso de listas pequeñas, los métodos directos se muestran eficientes, sobre todo porque los algoritmos son sencillos; su uso es muy frecuente. Sin embargo, en listas grandes estos métodos se muestran ineficaces y es preciso recurrir a los métodos avanzados.

12.3. ORDENACION POR BURBUJA

Este método es clásico y muy sencillo, aunque por desgracia poco eficiente. La *ordenación por burbuja* ("bubble sort") se basa en la comparación de elementos adyacentes de la lista (vector) e intercambiar sus valores si están desordenados. De este modo se dice que los valores más pequeños *burbujean* hacia la parte superior de la lista (hacia el primer elemento), mientras que los valores más grandes se *hundén* hacia el fondo de la lista.

Consideremos, como ya se ha expuesto, clasificaciones en orden creciente.

12.3.1. Análisis

Supongamos un vector $A[1], A[2], \dots, A[n]$. Se comienza el seguimiento del vector de izquierda a derecha, comparando $A[1]$ con $A[2]$; si están desordenados, se intercambian entre sí. A continuación se compara $A[2]$ con $A[3]$, intercambiándolos si están desordenados. Este proceso de comparaciones e intercambios continúa a lo largo de toda la lista. Estas operaciones constituyen una *pasada* a través de la lista. Al terminar esta pasada el elemento mayor está en la parte inferior de la lista y alguno de los elementos más pequeños ha burbujeado hacia arriba de la lista. Se vuelve a explorar de nuevo la lista, comparando elementos consecutivos e intercambiándolos cuando estén desordenados, pero esta vez el elemento mayor no se compara, ya que se encuentra en su posición correcta. Al terminar esta pasada se habrá situado en su sitio el segundo elemento más grande. Se siguen las comparaciones hasta que toda la lista está ordenada, cosa que sucederá cuando se hayan realizado $(n-1)$ pasadas. Para su mejor comprensión, veamos

gráficamente el proceso anterior con un vector (lista) de cinco elementos: A[1], A[2], A[3], A[4], A[5].

A[1]	23
A[2]	19
A[3]	45
A[4]	31
A[5]	15

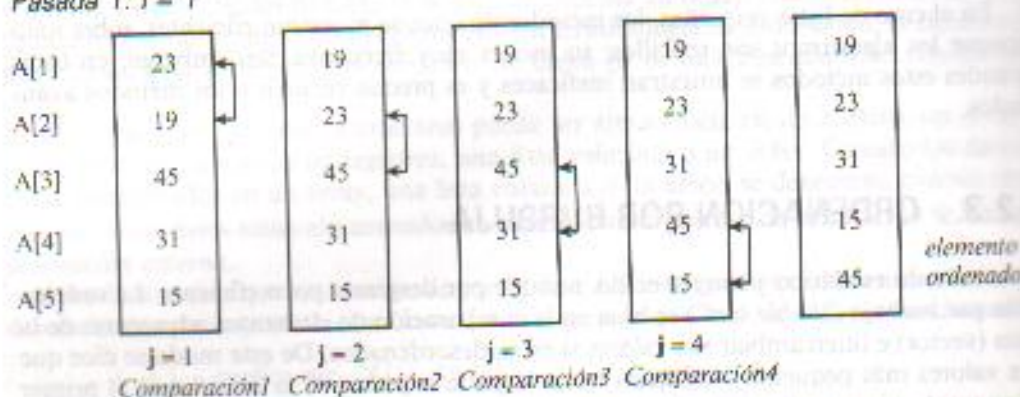
Lista sin ordenar

15
19
23
31
44

Lista ordenada

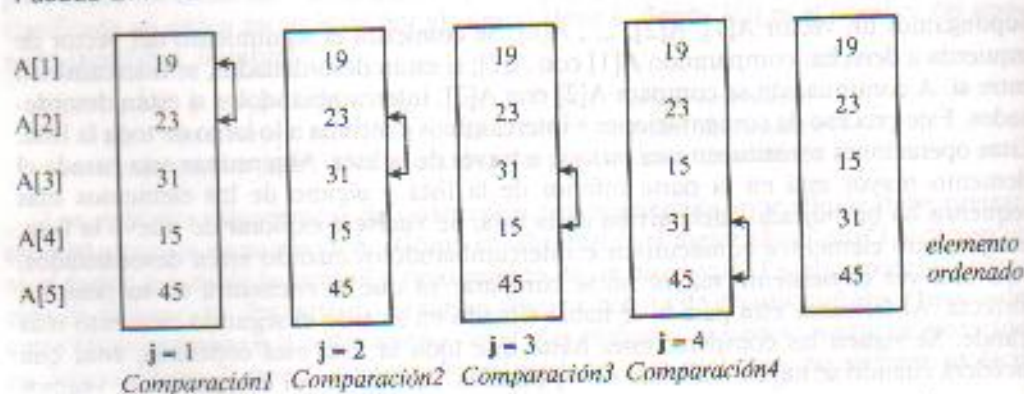
En la lista A, i será el número de la pasada y j indica el orden del elemento de la lista. Se comparan el elemento j -ésimo y el $(j + 1)$ -ésimo.

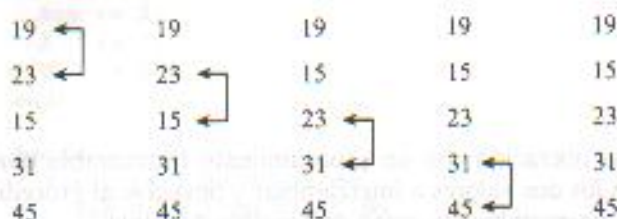
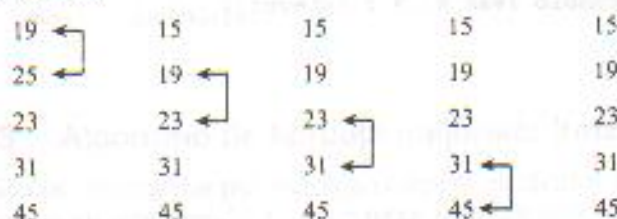
Pasada 1: $i = 1$



Se han realizado cuatro comparaciones ($5 - 1$ o bien $n - 1$, en el caso de n elementos) y tres intercambios (rotulados por el símbolo $\leftrightarrow$).

Pasada 2: $i = 2$



Pasada 3: $i = 3$ Pasada 4: $i = 4$ 

Se observa que se necesitan cuatro pasadas para ordenar una lista de números de cinco elementos, por lo que una lista de n elementos necesitará $n - 1$ pasadas. El proceso se describe así:

1. Realizar cuatro pasadas por la lista: $i = 1, 2, 3, 4$.
2. Para la pasada 1 ($i = 1$) se realizan comparaciones ($j = 1, 2, 3, 4$). $A[1]$ con $A[2]$, $A[2]$ con $A[3]$, etc.
3. Para la pasada 2 ($i = 2$) se realizan $3(5 - 3)$ comparaciones ($j = 1, 2, 3$).
4. Para la pasada 3 ($i = 3$) se realizan $2(5 - 3)$ comparaciones ($j = 1, 2$).
5. Para la pasada 4 ($i = 4$) se realizan $1(5 - 4)$ comparaciones ($j = 1$).

El número de pasadas (4 o bien $n - 1$) se puede controlar con un bucle **for**, y cada secuencia de comparaciones se puede controlar con un bucle **for** anidado al bucle de pasadas, en el que j varía desde 1 hasta 5 menos el valor específico de i .

<i>bucle externo</i>	$i = 1$	$j = 1, 2, 3, 4$	$5 - i = 4$	<i>bucle interno ($n - i$)</i>
i	$i = 2$	$j = 1, 2, 3$	$5 - i = 3$	
	$i = 3$	$j = 1, 2$	$5 - i = 2$	
	$i = 4$	$j = 1$	$5 - i = 1$	

Por consiguiente, el bucle **for** que controla cada pasada será:

```
for j = 1 to 5-i
```

Algoritmo (Pseudocódigo)

```
desde i ← 1 hasta n-1 hacer
  desde j ← 1 hasta n-i hacer
    si  $A[j] > A[j+1]$ 
      entonces Intercambio ( $A[j]$ ,  $A[j+1]$ )
  fin_si
fin_desde [bucle j]
fin_desde [bucle i]
```

La operación de intercambio se realiza con las instrucciones

```
Aux ← A[J]
A[J] ← A[J+1]
A[J+1] ← Aux
```

o mejor modularizando esta operación con un procedimiento Intercambio que recibe como parámetro de entrada los dos valores a intercambiar y devuelve al procedimiento llamador los dos valores ya intercambiados como parámetros de salida.

```
procedure Intercambio (var A, B : integer);
var
  Aux : integer;
begin
  Aux := A;
  A := B;
  B := Aux;
end;
```

12.3.2. Procedimiento de ordenación

Método 1

```
procedure burbujal (var A : lista; N : integer);
{ordenar A[1],...,A[N] en orden ascendente
 Lista es un array unidimensional definido en el programa
 principal de N elementos de tipo entero}
var
  I, J, Aux : integer;
begin
  for I := 1 to N - 1 do
    for J := 1 to N - I do
      if A[J] > A[J + 1] then
        begin {intercambio dentro del programa}
          Aux := A[J];
          A[J] := A[J + 1];
          A[J + 1] := Aux;
        end; {fin de los bucles for}
    end;
end; {Burbujal}
```

Método 2

```
procedure Burbujal (var A : lista; N : integer);
var
  I, J : integer;

procedure Intercambio (var X, Y : integer);
var
  Aux : integer;
```

```

begin
  Aux := X;
  X := Y;
  Y := Aux
end;

begin
  for I := 1 to N - 1 do
    for J := 1 to N - I do
      if A[J] > A[J + 1] then
        Intercambio (A[J], A[J + 1])
    end;
  (Burbuja)
end;

```

12.3.3. Algoritmo de burbuja mejorado (refinamiento)

La técnica de ordenación por burbuja compara elementos consecutivos de la lista, de modo que si en una pasada no ocurrieran intercambios, significaría que la lista está ordenada. El algoritmo burbuja se puede mejorar si disponemos de algún tipo de indicador que registre si se han producido intercambios en la pasada. Cuando se explore la lista y el indicador no refleje intercambios, la lista estará ya ocupada y se terminarán las comparaciones.

El indicador será una variable lógica NoIntercambio (o bien ordenado) que se inicializa a *true* (significa que la lista a priori está desordenada). Si dos elementos se intercambian en una pasada, NoIntercambio se pone a *false*. Al principio de cada pasada NoIntercambio se fija a *true* y a *false* si se produce a intercambios. El bucle externo *for* se sustituye por un bucle *repeat-until* o bien *while-do* y un contenido *i* se necesitará para contar el número de pasadas.

Pseudocódigo BurbujaMejorado

```

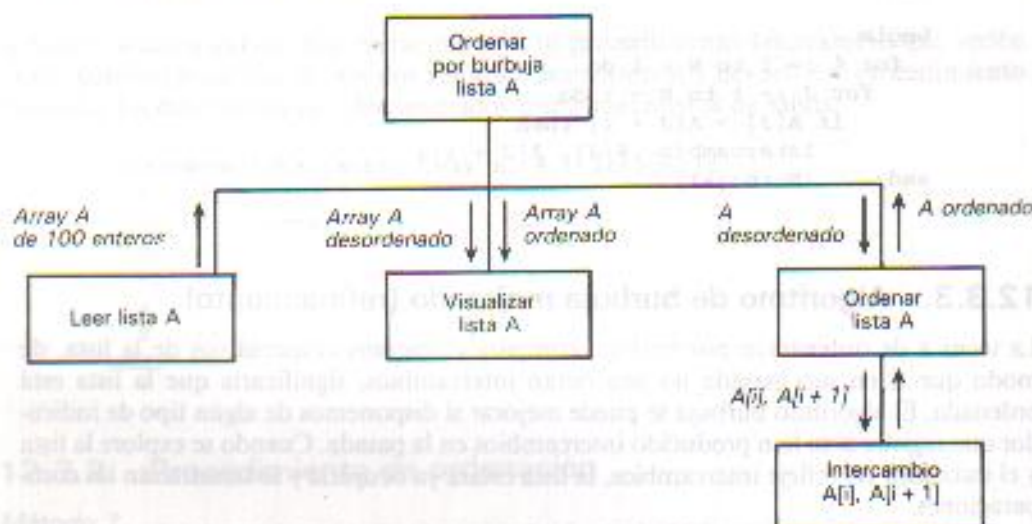
i ← 1
repetir
  NoIntercambio ← true
  desde j ← i hasta n-i hacer
    si A[j] > A[j+1]
      entonces Intercambio (A[j], A[j+1])
      NoIntercambio ← false
  fin-si
  fin-desde
  i ← i+1
hasta-que NoIntercambio = true

```

12.3.4. Programa completo de ordenación por burbuja

Se genera aleatoriamente una lista de 100 números enteros (o bien se leen de un archivo de entrada: teclado o disco) y se desea escribir un programa que realice las siguientes tareas:

1. Leer lista de números aleatorios.
2. Visualizar lista.
3. Ordenar lista por burbuja.
4. Visualizar lista ordenada.



El procedimiento Leer sirve para introducir la lista de 100 números; el procedimiento Visualizar permite imprimir cualquier lista, en este caso tanto la lista ordenada como desordenada; y el procedimiento Ordenar clasifica la lista A.

```

program OrdenarBurbuja;
{ Ordenación ascendente por el método de la burbuja }
const
  Limite = 100;
type
  Item = integer;
  rango = 0..Limite;
  Lista = array [Rango] of Item;
var
  ListaItem : Lista;
  Numitems : integer;

procedure Leer (var A: Lista; N : integer);
var
  I : integer;
begin
  for I := 1 to N do
    A [I] := Random (1000)
  end;

procedure Escribir (var A: Lista; N : integer);
var
  I : integer;

```

```

begin
  for I := 1 to N do
    Write (A[I] : 4);
  WriteLn
end;

procedure Burbuja (var A: Lista; N : integer);
var
  I, J : integer;
  NoIntercambio : boolean;
  { procedimiento de intercambio }
  procedure Intercambio (var A, B : item);
  var
    Aux : item;
  begin
    Aux := A;
    A := B;
    B := Aux;
  end;
begin
  for I := 1 to N - 1 do
    for J := I + 1 to N do
      if A[I] > A[J]
      then
        Intercambio (A[I], A[J]);
    end;
  end;
end;

{ programa principal }
begin
  Leer (ListaItem, Limite);
  WriteLn ('La lista original es ');
  Escribir (ListaItem, Limite);
  Burbuja (ListaItem, Limite);
  WriteLn ('La lista ordenada es ');
  Escribir (ListaItem, Limite);
end.

```

12.3.5. Análisis de la ordenación por burbuja

Este algoritmo proporciona buen rendimiento en cuanto a su sencillez, pero por el contrario su eficiencia es pobre. Para una lista de N elementos, el proceso de ordenación requiere $N-1$ pasadas y el número de comparaciones se refleja en la tabla siguiente:

Pasada	Comparaciones
1	$N-1$
2	$N-2$
3	$N-3$
.	.
.	.
$N-1$	1

El número total de comparaciones es

$$1 + 2 + 3 + \dots + (N - 3) + (N - 2) + (N - 1) = \frac{N^2 - N}{2} = \frac{1}{2} (N^2 - N)$$

La función de eficiencia de rendimiento de un algoritmo se representa con la función $O(N)$, también llamada *Notación de O-grande*. En el caso de la burbuja, en el peor de los casos, el número de comparaciones es $O(N^2)$, y en el mejor de los casos es $O(N)$. El peor de los casos se da cuando el vector está invertido (ordenado por orden descendente de claves, el número de intercambios es $O(N^2)$. En el mejor de los casos, el número de intercambios depende de N , es decir, $O(N)$.

El algoritmo de burbuja es una ordenación cuadrática, lo que significa elevado número de comparaciones y, por consiguiente, excesivo tiempo de ejecución, o dicho de otro modo: es un *algoritmo lento*.

12.4. ORDENACION POR SELECCION

El algoritmo de ordenación por selección de una lista (vector) de n elementos tiene los siguientes pasos:

1. Encontrar el elemento mayor de la lista.
2. Intercambiar el elemento mayor con el elemento de subíndice n (o bien el elemento menor en el subíndice 1).
3. A continuación se busca el elemento mayor en la sublista de subíndices $1 \dots n - 1$, y se intercambia con el elemento de subíndice $n - 1$; por consiguiente, se sitúa el segundo elemento mayor en la posición $n - 1$.
4. A continuación se busca el elemento mayor en la sublista $1 \dots n - 2$, y así sucesivamente.

Algoritmo

Desde $j \leftarrow n$ hasta 2 [decremento -1] hacer:

- Encontrar el elemento mayor en el array $1 \dots j$.
- Si el elemento mayor no está en el subíndice j , entonces intercambiar elemento mayor con el de subíndice j .

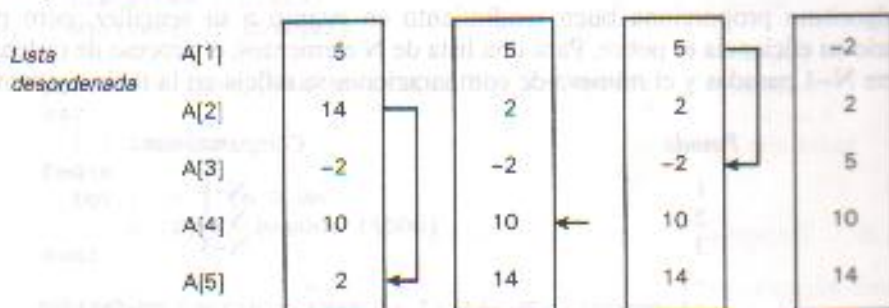


Figura 12.1. Seguimiento de la ordenación por selección.

El algoritmo de PosMayor debe guardar j como la posición del elemento mayor y luego poder intercambiar.

```

program OrdenarSeleccion;
{ leer 100 enteros. Ordenar. Visualizar }
const
  Limite = 100;
type
  Lista = array [1..Limite] of integer;
var
  I, Num : 1..Limite;
  A      : Lista;

function PosMayor (Ultimo : integer; var Tabla: Lista) : integer;
{ encuentra el indice del elemento mayor en la Tabla [1..Ultimo]}
var
  Indice_Max, Indice : 1..Limite;
begin
  Indice_max := 1;
  for Indice := 2 to Ultimo do
    if Tabla [Indice] > Tabla [Indice_Max]
    then Indice_Max := Indice;
  PosMayor := Indice_Max
end;

procedure Seleccion (Limi : integer; var Tabla : Lista);
var
  Aux, J, Mayor : integer;
begin
  for J := Limi downto 2 do
    begin
      { encontrar el Elemento mayor de 1..J }
      Mayor := PosMayor (J, Tabla);
      { intercambio con el Elemento Tabla [J] }
      Aux      := Tabla [Mayor];
      Tabla [Mayor] := tabla [J];
      Tabla [J]    := Aux
    end
  end;

{ programa principal }
begin
  for I := 1 to Limite do
    begin
      A [I]      := Random (100);
      Write (A[I] :4);
    end;
  WriteLn;
  Seleccion (Limite, A);
  for I := 1 to Limite do
    Write (A[I]:4);
  WriteLn
end.

```

Análisis de la ordenación por selección

Número de comparaciones por cada una de las pasadas.

<i>Pasada</i>	<i>Número de comparaciones</i>
1	N-1
2	N-2
3	N-3
⋮	⋮
N-1	1

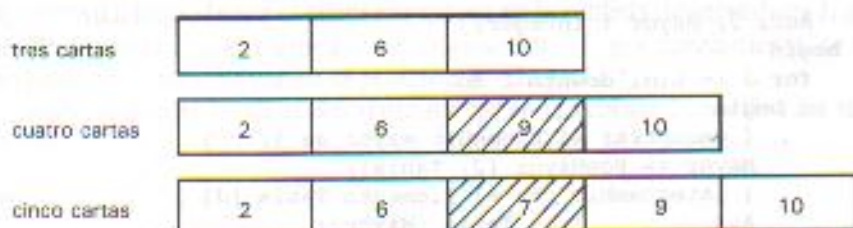
El número total de comparaciones es

$$1 + 2 + 3 + \dots + N - 2 + N - 1 = \frac{N \cdot (N - 1)}{2} = \frac{1}{2} (N^2 - N)$$

La eficiencia, como se observa, es similar al método de la burbuja.

12.5. ORDENACION POR INSERCIÓN

Este método está basado en la técnica utilizada por los jugadores de cartas para clasificar sus cartas. El jugador va colocando (insertando) cada carta en su posición correcta.



El método se basa en considerar una parte de la lista ya ordenada y situar cada uno de los elementos restantes insertándolo en el lugar que le corresponde por su valor.

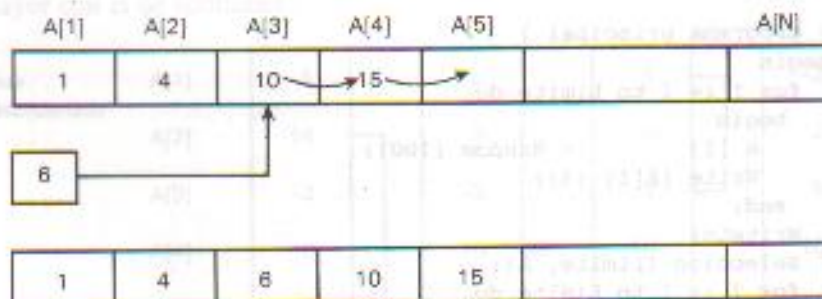


Figura 12.2. Ejemplo de ordenación por inserción: 6 se inserta justo antes de 10, y todos los valores a la derecha se desplazan una posición para dejar espacio.

Algoritmo

```

[para cada elemento de la lista despues del primero]
desde k ← 2 hasta n hacer
    • Guardar el valor de este elemento A[k] en una variable Aux.
    • Hacer espacio para Aux desplazando todos los valores mayores que dicho valor A[k] una posición.
    • Insertar el valor de Aux en el lugar del ultimo valor desplazado.
fin_desde

```

La operación de desplazamiento se realiza con un procedimiento Desplazar, que mueve todos los elementos de la lista mayores que Aux, comenzando con el elemento de la lista de posición Aux-1. Si Aux es el valor más pequeño hasta aquí, la operación de desplazamiento termina después que todos los elementos de la lista se han desplazado. Si Aux no es el valor más pequeño, la operación de desplazamiento termina cuando un valor menor o igual a Aux se alcanza. Aux se inserta en la posición que ocupaba el último valor que se desplazó.

Algoritmo de desplazamiento

```

mientras el primer elemento no se desplaza y valor del elemento > Aux hacer
    • Desplazar elemento una posición.
    • Comprobar valor del siguiente elemento.
    • Definir NuevaPos como posición original del ultimo elemento desplazado.
fin_mientras

```

Codificación del procedimiento OrdenarInserción

```

procedure OrdenacionInversa (var Tabla: Lista; N: integer);
{ Tabla (entrada/salida) , N (entrada) }
{ Lista = array de N elementos enteros }
var
    K      : integer; { subíndice del siguiente elemento al
                       que se inserta }
    NuevaPos : integer; { subíndice de este elemento despues
                       de la insercion }
    Aux     : integer;

begin
    for K := 2 to N do
        begin
            Aux := Tabla [K]; { obtener siguiente elemento a
                              insertar }
            { desplazar todos los valores > Aux un elemento }
            Desplazar (Tabla, K , Aux, NuevaPos);
            { insertar Aux en posición NuevaPos }
            Tabla [NuevaPos] := Aux;
        end
    end;
end;

```

El procedimiento Desplazar es

```

procedure Desplazar (var Tabla : Lista; Aux, K : integer;
                    var NuevaPos : integer);
var
    Encontrado : boolean; {indicador}
begin
    {desplazar valores > Aux . Comenzar con el elemento K - 1}
    Encontrado := false;
    while (K > 1) and (not Encontrado) do
        if (Tabla [K - 1] > Aux) then
            begin
                Tabla [K] := Tabla [K - 1];
                K := K - 1;
            end
        else
            Encontrado := true;
            NuevaPos := K;
        end;
    {Desplazar}

```

12.5. ORDENACIÓN POR INSERCIÓN

12.5.1. Algoritmo de inserción binaria

El algoritmo de inserción se mejora si en lugar de realizar una búsqueda secuencial o lineal se realiza una búsqueda binaria. En el apartado 12.7 se analizan los algoritmos de búsqueda y sus dos métodos clásicos: secuencial y binaria.

12.5.2. Análisis de la ordenación por inserción

El análisis de la ordenación por inserción es un poco más complicado. El número de comparaciones en el i -ésimo paso es como máximo $k - 1$ y como mínimo 1. Por consiguiente, la media proporciona el número de comparaciones.

$$[(k-1) + 1] / 2 = k/2$$

El número de comparaciones (c) es

$$C_{\max} = \sum_{i=2}^N (i-1) = 1 + 2 + \dots + (N-1) = \frac{N^2 - N}{2} = \frac{1}{2} (N^2 - N)$$

$$C_{\min} = \sum_{i=2}^N (1) = 1 + 1 + \dots + 1 = (N-1)$$

$$C_{\text{media}} = (C_{\max} + C_{\min})/2 = \left(\frac{1}{2} (N^2 - N) + (N-1) \right) / 2 = \frac{1}{4} (N^2 + N - 2)$$

Como se observa, la eficiencia es $O(n^2)$.

12.5.3. Comparación de ordenaciones cuadráticas

	Caso favorable	Caso desfavorable
Selección	$O(N^2)$	$O(N^2)$
Burbuja	$O(N^2)$	$O(N^2)$
Inserción	$O(N^2)$	$O(N^2)$

Como el tiempo requerido para ordenar un array (vector) de n elementos es proporcional a N^2 , ninguno de estos algoritmos es particularmente bueno para arrays grandes ($N \geq 100$). Para listas de mayor número de elementos, los métodos avanzados son los más idóneos ya que su eficiencia en lugar de depender de N^2 depende de $N \times \log_2 N$, lo que reduce considerablemente el tiempo de ejecución.

12.6. ORDENACION SHELL

La ordenación Shell debe el nombre a su inventor, D. L. Shell [CACM 2 (Julio, 1959), 30-32]. Se suele denominar también *ordenación por disminución de incremento (gap)*. La idea general del método (algoritmo) es la siguiente:

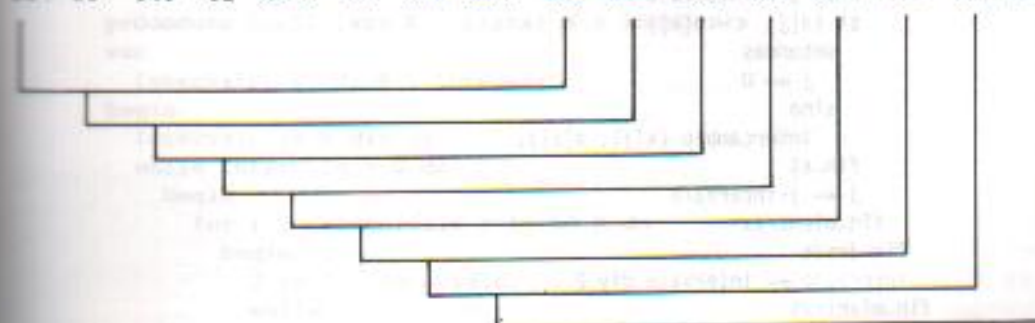
Lista original

504 88 513 62 908 171 898 277 654 427 150 510 612 675 750 704

1. Se divide la lista original (16 elementos, en este ejemplo) en ocho grupos de dos (considerando un incremento o intervalo de $16/2 = 8$).
2. Se clasifica cada grupo por separado (se comparan las parejas de elementos y si no están ordenados se intercambian entre sí de posiciones).
3. Se divide ahora la lista en cuatro grupos de cuatro (intervalo o salto de $8/2 = 4$) y nuevamente se clasifica cada grupo por separado.
4. Un tercer paso clasifica dos grupos de ocho registros y luego un cuarto paso completa el trabajo clasificando todos los 16 registros.

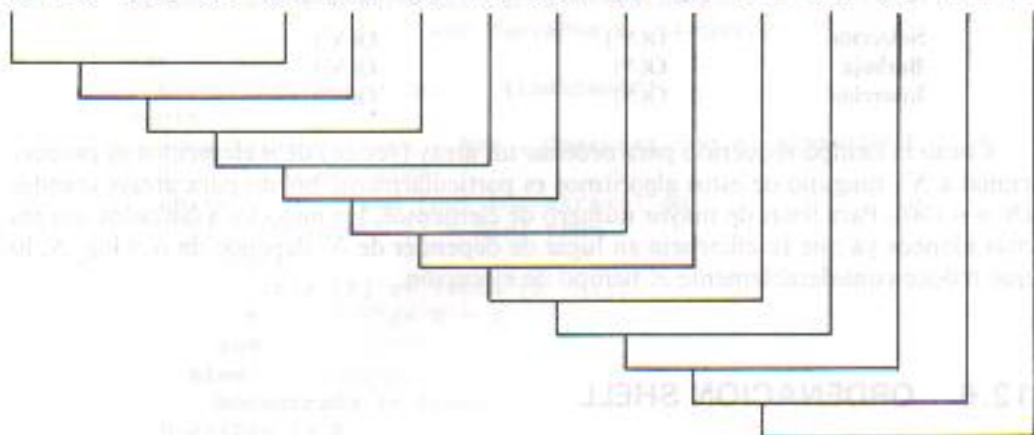
Primer paso (división/ordenación por 8)

504 88 513 62 908 171 898 277 654 427 150 510 612 675 760 704



Segundo paso (división/ordenación por 4)

504 88 150 62 612 171 760 277 654 427 513 510 908 675 898 704



Tercer paso (división/ordenación por 2)

504 88 150 62 612 171 513 277 654 427 760 510 908 675 898 704



Cuarto paso (división/ordenación por 1)

154 62 504 88 513 171 612 277 654 427 760 510 898 675 908 704



62 88 154 171 277 427 504 510 513 612 654 675 704 760 898 908

El algoritmo de Shell tiene diferentes modelos; recogemos en este libro uno de los más populares y citados en numerosas obras de programación.

Algoritmo

```

intervalo ← n div 2
mientras (intervalo > 0) hacer
  desde i ← (intervalo + 1) hasta n hacer
    j ← i - intervalo
    mientras (j > 0) hacer
      k ← j + intervalo
      si (a[j] <= a[k])
        entonces
          j ← 0
        sino
          Intercambio (a[j], a[k]);
      fin-si
      j ← j - intervalo
    fin-mientras
  fin-desde
  intervalo ← intervalo div 2
fin-mientras

```

Programa

```

program OrdenarShell;
{ modelo de ordenacion de 500 enteros aleatorios }
const
  NumElementos = 500;
type
  Rango = 1..NumElementos;
  Lista = array [Rango] of integer;
var
  L : Lista;

procedure GenerarAleatorios (var A:Lista; Elementos : integer);
var
  I : integer;
begin
  randomize;
  for I := 1 to Elementos do
    A [I] := Random (1000)
  end;

procedure Visualizar (var A: Lista; Elementos : integer);
var
  I : integer;
begin
  for I := 1 to Elementos do
    Write (A[I] :6, ' ');
  WriteLn
end;

procedure Intercambio (var X,Y : integer);
var
  Aux : integer;
begin
  Aux := X;
  X := Y;
  Y := Aux
end;

procedure Shell (var A : Lista; N : integer);
var
  Intervalo, I, J, K : integer;
begin
  Intervalo := N div 2;
  while Intervalo > 0 do
    begin
      for I := (Intervalo + 1) to N do
        begin
          J := I - Intervalo;
          while (J > 0) do

```

```

begin
  K := J + Intervalo;
  if A [J] <= A [K]
  then
    J := 0
  else
    Intercambio (A[J], A[K]);
    J := J - Intervalo
  end {while}
end;
Intervalo := Intervalo div 2
end
end;

begin
  { programa principal }
  WriteLn ('Comienza la ordenacion ');
  GenerarAleatorios (L, NumElementos);
  Shell (L, NumElementos);
  Visualizar (L, NumElementos)
end.

```

12.7. BUSQUEDA LINEAL

Otro problema importante en proceso de datos, como ya se ha comentado, es la *búsqueda* en un conjunto de datos de un elemento específico y la recuperación de alguna información asociada al mismo.

Existen diferentes métodos de búsqueda:

búsqueda lineal o secuencial
búsqueda binaria o dicotómica
búsqueda hash o por conversión de claves

Las dos primeras se pueden aplicar a listas implementadas con arrays, y la tercera es más propia de estructuras tipo registros o archivos. En esta sección y en la siguiente trataremos de la búsqueda lineal, dejando para el capítulo de registros el tercer método.

12.7.1. Análisis

La búsqueda lineal o secuencial es la técnica más simple para buscar un elemento en un array (vector). Consiste en el método en el recorrido de todo el vector, desde el primer elemento hasta el último, y de uno en uno. Si el vector contiene el elemento, el proceso devolverá la posición del elemento buscado dentro del vector y, en caso contrario, un mensaje que indique la falta de éxito en la búsqueda.

Ejemplo

Supongamos una lista de números de la Seguridad Social incluidos en un array a y se desea buscar a ver si existe el número 453714.

$a[1]$	451871
$a[2]$	120467
$a[3]$	401321
$a[4]$	25761
$a[98]$	339412
$a[99]$	81467
$a[100]$	924116

Números Seguridad Social

elemento a buscar: t
 453714

Figura 12.3. Búsqueda lineal de un elemento.

Mediante un bucle desde ir comparando el elemento t buscado con $a[i]$, donde i varía, en el ejemplo anterior, de 1 a 100. En caso de encontrarlo, almacenar la posición (el índice de array) del mismo dentro de la lista y finalmente se devolverá al programa principal. Dado que los algoritmos de búsqueda normalmente sólo devuelven la posición, es muy frecuente que la implementación del algoritmo se haga con una función.

Pseudocódigo 1

```

Posición ← 0
| lista = vector  $a[i]$  de  $n$  elementos
desde  $i \leftarrow 1$  hasta  $n$  hacer
    si  $a[i] = t$ 
        entonces Posición ←  $i$ 
    fin-si
fin-desde
  
```

Este algoritmo tiene un grave inconveniente: sea cual sea el resultado (existe/no existe el elemento) se recorre el vector completo. El algoritmo tiene una mejora: detectar el momento de localizar el elemento y terminar el bucle. Así el algoritmo mejorado se puede realizar con un bucle **while** o **repeat**, y utilizando unas banderas (interruptor) que detecten cuándo se encuentra el elemento. El bucle se terminará por dos causas:

- La bandera o indicador (Encontrado, por ejemplo) toma el valor esperado (por ejemplo, *true* —verdadero—) y la búsqueda ha tenido éxito.
- El valor del índice *i* es mayor que el número de términos de la lista, lo que significa que se ha terminado de recorrer la misma y el elemento buscado no ha aparecido.

Pseudocódigo 2

```

Encontrado ← falso
Posición ← 0
i ← 1
mientras (i ≤ n) y (No Encontrado=verdadero) hacer
  si a[i] = t
    entonces Posición ← i
    Encontrado ← verdadero
  fin-si
  i ← i + 1
fin-mientras

```

o bien con la estructura **repetir**

```

repetir
  ...
hasta-que (Encontrado=verdadero) o (i > n)

```

Función búsqueda lineal

```

function BusquedaLineal (A: Lista; {entrada, vector busqueda}
N : integer; {entrada, numero de elementos}
T : integer; {elemento a buscar}) : integer
var
  Encontrado : boolean;
  I : integer;
begin
  BusquedaLineal := 0; {posicion del elemento caso de no existir}
  Encontrado := false;
  I := 1;
  while (I ≤ N) and not Encontrado do
    if A[I] = T then
      begin
        BusquedaLineal := I;
        Encontrado := true;
      end; {fin del if y del while}
    I := I + 1
  end {posicion del elemento en la lista}

```

Si se desea saber en el programa principal si existe el elemento, bastará preguntar con una sentencia *if* cuál es el valor de la función *BusquedaLineal*; si es cero no existe el elemento, y en caso contrario existe y su valor es la posición en la lista o vector.

Nota

Como Turbo Pascal tiene la sentencia **exit** para terminación anormal de un bucle, la búsqueda lineal se podría terminar nada más encontrarse el elemento ejecutado a continuación de la sentencia **exit** (en general, el uso de **exit** está orientado a la programación en tiempo real).

```
function BusquedaLinealDos (A: Lista; N : integer;
                           T: integer ): integer;

var
  I : integer;
begin
  for I := 1 to N do
    if T = A[I] then
      begin
        BusquedaLineal := I;
        Exit
      end;
  BusquedaLineal := 0
end;
```

Programa (se muestra una variante del algoritmo de la función)

```
program LinealBusqueda;
const
  Total = 100;
type
  Lista = array [1..Total] of integer;
var
  L      : Lista;
  P, J, Num : integer;

function BusquedaLineal (t : integer; A: Lista; Maximo: integer) :
                           integer;

var
  I      : integer;
  Encontrado : boolean;
begin
  Encontrado := false;
  I := 0;
  while (I < Maximo) and not Encontrado do
    begin
      I := I + 1;
      Encontrado := A [I] = t
    end;
  if Encontrado
  then BusquedaLineal := I
  else BusquedaLineal := 0
end;
```

```

begin { programa principal }
for J := 1 to Total do { lectura de 100 enteros aleatorios }
  L [J] := Random (100);
repeat
  Write ('introduzca numero a buscar ');
  ReadLn (Num);
  P := BusquedaLineal (Num, L, Total);
  if P = 0
  then
    WriteLn ('no existe el numero en la lista')
  else
    WriteLn ('encontrado en la posicion ', P);
until Num = 0 { marca fin de datos de entrada };
end.

```

12.7.2. Eficiencia de la búsqueda lineal

El método de búsqueda secuencial, en el peor de los casos (el elemento buscado está al final de la lista o no existe), requiere consultar los n elementos de la lista para encontrar el elemento deseado o determinar que el elemento no existe en la lista. Entonces el tiempo de búsqueda es directamente proporcional al número de elementos de la lista, por lo que utilizando la notación O se tiene para el tiempo t la fórmula:

$$t = O[f(n)]$$

o simplificando

$$t = O(n)$$

12.8. BUSQUEDA BINARIA

La búsqueda lineal, por su simplicidad, es buena para listas de datos pequeñas. Para listas grandes es ineficiente; la búsqueda binaria es el método idóneo. Se basa en el conocido método de *divide y vencerás*.

Este método tiene una clara expresión en la búsqueda de una palabra en un diccionario. Cuando se busca una palabra no se comienza la búsqueda por la página 1 y se sigue secuencialmente, sino que se abre el diccionario por una página donde aproximadamente se piensa puede estar la palabra, es decir, se divide el diccionario en dos partes; al abrir la página se ve si se ha acertado o en qué parte (la primera o la segunda) se encuentra la palabra buscada. Se repite este proceso hasta que por divisiones o aproximaciones sucesivas se encuentra la palabra.

Supongamos que la lista donde se busca es

	1331	
	1373	
	1555	
	1850	
	1892	
	1898	← elemento central
elemento buscado →	1899	
	2002	
	2400	
	2670	
	3200	

y que se busca el número 1899.

Se examina en primer lugar el elemento central de la lista (las divisiones se toman iguales), 1898. Dado que 1899 es mayor que 1898, el elemento a buscar estará en la segunda mitad. Por consiguiente, se sigue la búsqueda en esta mitad:

elemento a buscar →	1899	
	2002	
	2400	← elemento central
	2670	
	3200	

El elemento central en esta sublista es 2400, y como 1899 es menor, la nueva sublista donde buscar es

1899	← elemento considerado central
2002	

Como ya no hay elemento central, se toma el número inmediatamente anterior a la posición central, que en este caso es 1899. En este caso se ha encontrado el elemento deseado en tres comparaciones, mientras que en la búsqueda lineal hubiese necesitado al menos seis comparaciones (la mitad de los elementos, redondeada a un entero).

Este método es muy eficiente, con el único inconveniente, como habrá deducido, de requerir la lista ordenada.

Algoritmo

1. Establecer $\text{Primer} \leftarrow 1$ y $\text{Ultimo} \leftarrow n$ (n , número de elementos). Estas variables representan la primera y última posición de la lista o sublista donde se está buscando y permite el cálculo de la posición del elemento central.
2. $\text{Encontrado} \leftarrow \text{falso}$ (variable lógica).
3. mientras $\text{Primer} < \text{Ultimo}$ y $\text{Encontrado} = \text{falso}$ hacer
 - {Encontrar posición central}
 - $\text{Central} \leftarrow (\text{Primer} + \text{Ultimo}) \div 2$
 - {Comparar elemento buscado t con $a[\text{Central}]$ }
 - si $t = a[\text{Central}]$
 - entonces $\text{Encontrado} \leftarrow \text{verdadero}$

```

    sino si t > a[Central]
        entonces Primero ← Central + 1
        sino Ultimo ← Central - 1
    fin_mientras
4. si Encontrado = verdadero
    entonces Posicion ← Central {existe el elemento}
    sino Posicion ← 0 {no se ha encontrado}
fin-si

```

Nota

Obsérvese que *Encontrado = falso* verdadero equivale a *Not Encontrado* cuando se ha inicializado *Encontrado* a falso.

Programa

La búsqueda binaria requiere una ordenación previa del vector o lista en el que se va a efectuar la búsqueda. Por consiguiente, las acciones típicas (módulos) en un algoritmo de búsqueda binaria son:

1. Lectura del vector.
2. Ordenación del vector.
3. Búsqueda binaria.
4. Visualizar resultados.

Aprovechando que en Turbo Pascal disponemos de la directiva de compilación **SI**, compile el procedimiento de ordenación Shell, de la forma explicada en el Capítulo 6, para archivos de inclusión y dele el nombre Shell, se grabará con Shell.inc.

```

program BusquedaBinaria;
const
    Limite = 100;
type
    Lista = array [1..Limite] of integer;
var
    A : Lista;
    I, J, t : integer;
    { directiva de inclusion, aquí se inserta el procedimiento
      Shell }
    { si se ha grabado en disco como se indica en el capítulo 6 }
    {$I Shell.inc}
function Binaria (T : integer; var L : Lista; N : integer) : integer;
var
    Primero, Ultimo, Central : integer;
    Encontrado : boolean;
begin
    Primero := 1;
    Ultimo := N;
    Encontrado := false;
    while (Primero <= Ultimo) and not Encontrado do

```

```

begin
  Central := (Primero + Ultimo) div 2;
  if T = L[Central]
  then
    Encontrado := true
  else
    if T > L[Central]
    then Primero := Central + 1
    else Ultimo := Central - 1
  end;
  { while }
if not Encontrado
then Binaria := 0
else Binaria := Central
end;

begin { programa principal }
{ lectura de 100 enteros aleatorios }
for I := 1 to 100 do
  A[I] := Random (100);
Shell (A, 100);
Write ('Introduzca numero a buscar');
ReadLn (T);
J := Binaria (T, A, 100);
if J = 0
then WriteLn ('el numero no figura en lista')
else WriteLn ('el numero ocupa la posicion ', J);
WriteLn
{ Si desea repetir la búsqueda, añada a este programa una
estructura repetitiva }
end.

```

12.8.1. Eficiencia de la búsqueda binaria

En el algoritmo de búsqueda binaria, con cada comparación se divide en dos mitades el tamaño de la lista en estudio. Si n es el tamaño de la lista, los tamaños sucesivos de las sublistas serán

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots$$

$$\frac{n}{2^1}, \frac{n}{2^2}, \frac{n}{2^3}, \dots$$

El proceso terminará cuando el tamaño se hace igual a menor a 1. Por consiguiente, si k es el número mayor de comparaciones

$$\frac{n}{2^k} < 1$$

$$n < 2^k$$

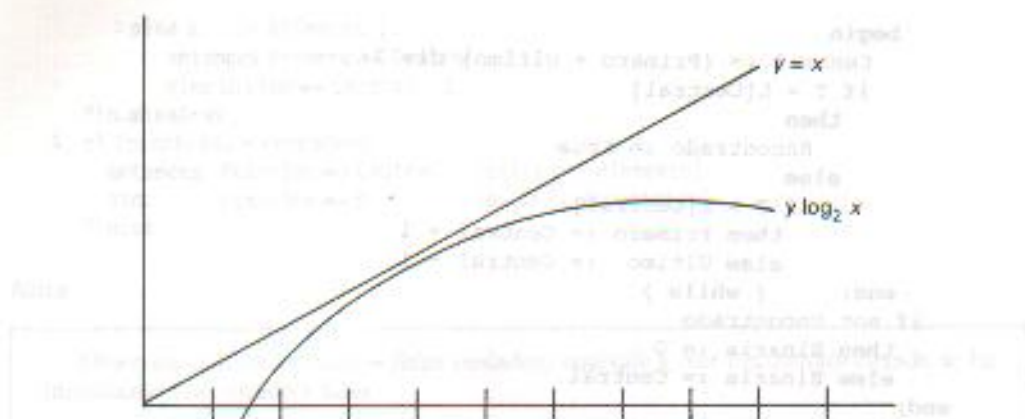


Figura 12.4. Funciones O de búsqueda.

Si se toman logaritmos en base 1, en ambos lados, se tiene

$$\log_2 n < k \cdot \log_2 2 = k$$

$$\log_2 n < k$$

Por consiguiente, la eficiencia de la búsqueda binaria se puede escribir como

$$O(\log_2 n)$$

que es bastante más rápido que la búsqueda lineal, como se puede ver en la Figura 12.4, que representa las dos funciones O .

El tiempo que se ahorra utilizando el algoritmo de búsqueda binaria es muy considerable. Para una lista de 50.000 elementos, la búsqueda lineal en el peor de los casos requiere 50.000 comparaciones y 25.000 por término medio, mientras que la búsqueda binaria *nunca* requerirá más de $\log_2 50.000$, es decir, alrededor de dieciséis comparaciones. A fin de ser sinceros, al tiempo de la búsqueda binaria habría que sumarle el tiempo empleado en ordenar la lista.

Recuerde

La búsqueda binaria sólo funcionará correctamente si la lista está ordenada. Sin embargo, la búsqueda lineal funciona tanto si la lista está como si no está ordenada.

12.9. MEZCLA

El proceso de **mezcla**, **fusión** o **intercalación** (*merge* en inglés) consiste en tomar dos vectores ordenados (a , b) y obtener un nuevo vector (c) también ordenado.

Procedimiento mezcla de los vectores A y B

```

procedure Mezcla (var A,B,C : Lista; M,N : integer);
{ A Y B: entrada. Vectores ya ordenados }
{ M Y N: numero de elementos de A Y B respectivamente }
{ C : salida. Vector mezcla ordenado }
{ El tipo Lista, tendra una longitud minima de M + N elementos }
var
  I,J,K : integer;
begin
  I := 1;
  J := 1;
  K := 1;
  while (I <= M) and (J <= N) do
  begin
    if A[I] <= B[J]
    then
      begin
        C[K] := A[I];
        I := I + 1;
      end
    else
      begin
        C[K] := B[J];
        J := J + 1;
      end;
    K := K + 1;
  end;
  {copiar el resto del vector no agotado}
  if I > M
  then
    for P := J to N do
    begin
      C[K] := B[P];
      K := K + 1;
    end
  else {J > N}
    for P := I to M do
    begin
      C[K] := A[P];
      K := K + 1;
    end
  end;
end;

```

RESUMEN

En este capítulo se han tratado los algoritmos y programas correspondientes de ordenación, búsqueda y mezcla. Las consideraciones más importantes a tener en cuenta son: